

UNIVERSIDAD NACIONAL DE SAN JUAN
FACULTAD DE CIENCIAS EXACTAS FÍSICAS Y NATURALES
DEPARTAMENTO DE INFORMÁTICA

Asignatura: Teoría de Información

TRABAJO PRÁCTICO:
ÁREA TEMÁTICA N° 1: COMPACTADORES

Alumnos:

- Baltasar Ortiz, 21752
- Gonzalo Balderrama, E-010-7
- Alejandro Riveros, 16567
- Octavio Garelo, 21438

Práctico 3 - Primera Parte

Teoría de la Información 2026 — Licenciatura en Ciencias de la Computación · Ejercicios 1 a 7

Notación y método (apuntes de la cátedra): logaritmos en base 2 salvo indicación; $L = \sum p_i l_i$; eficiencia $\eta = H/L$; redundancia $R = 1 - \eta$; porcentaje de compresión = $(1 - S_{\text{comp}}/S_{\text{orig}}) \times 100$ tomando como referencia el *código bloque* de $\lceil \log_2 q \rceil$ bits/símbolo (no se cuenta el costo de tablas o cabeceras salvo que se indique). Punto decimal en todo el documento.

Convención de este documento. Para Shannon, Fano y Huffman solo se dan **valores finales** (códigos, L , η); el procedimiento se dibuja a mano. Los incisos marcados con **RESOLVER MANUALMENTE** se dejan sin desarrollo y solo traen el *resultado esperado* para verificar.

Interpretaciones del enunciado (marcadas con ⚠ en el texto)

#	Dónde	Interpretación adoptada
1	Práctico 3, ej. 2	La tabla indica cuántas palabras hay de cada longitud (encabezado 1...5 = longitudes) y el alfabeto es ternario {0, 1, 2}.

Ejercicio 1 — Clasificación de códigos

s_i	p_i	a	b	c	d	e	f
s1	1/2	000	0	0	0	0	0
s2	1/4	001	01	10	10	10	100
s3	1/16	010	011	110	110	1100	101
s4	1/16	011	0111	1110	1110	1101	110
s5	1/16	100	01111	11110	1011	1110	111

s6	1/16	101	011111	111110	1101	1111	001
----	------	-----	--------	--------	------	------	-----

Fuente: La entropía es una propiedad de la **fente** (las probabilidades p_i), no del código. Como los seis códigos (a, b, c, d, e, f) codifican la misma fuente de 6 símbolos, **la entropía es la misma para los seis: $H(S) = 2$ bits/símbolo**.

$$H(S) = \sum_i p_i \log_2(1/p_i)$$

$$H(S) = \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{16} \cdot 4 + \frac{1}{16} \cdot 4 + \frac{1}{16} \cdot 4 + \frac{1}{16} \cdot 4$$

$$H(S) = 0.5 + 0.5 + 0.25 + 0.25 + 0.25 + 0.25$$

$$H(S) = 2 \text{ bits/símbolo}$$

a) Clasificar cada uno de ellos (Bloque, No singular, Instantáneos, Unívocamente Decodif.)

Criterios:

- **bloque** = todas las palabras de igual longitud;
- **no singular** = palabras distintas;
- **instantáneo** = ninguna palabra es prefijo de otra;
- **unívocamente decodificable (UD)** = toda secuencia se descompone de una única manera (necesita $\sum 2^{-l_i} \leq 1$, desigualdad de Kraft-McMillan).

Código	Bloque	No singular	Instantáneo	UD	$\sum 2^{-l_i}$	Justificación
a	✓	✓	✓	✓	$6/8 = 0.75$	Longitud fija 3; queda libre 110 y 111.
b	✗	✓	✗	✓	$63/64 = 0.98$	Cada palabra es prefijo de la siguiente ($0 \subset 01 \subset 011 \dots$). Es UD: todas empiezan con 0 y no tienen otro 0, así que cada 0 abre una palabra nueva ; el decodificador debe esperar un bit más (retardo 1). O por Kraft-McMillan es UD.

c	x	✓	✓	✓	$63/64=0.98$	Código <i>coma</i> : el 0 cierra cada palabra. Es UD por desigualdad de Kraft-McMillan.
d	x	✓	x	x	$17/16 > 1$	10 es prefijo de 1011 y 110 lo es de 1101; viola McMillan. Ambigüedad: 11010 = s3 s2 = s6 s1.
e	x	✓	✓	✓	1	Árbol completo (Kraft con igualdad).
f	x	✓	x	x	$9/8 > 1$	0 es prefijo de 001; viola McMillan. Ambigüedad: 001100 = s6 s2 = s1 s1 s4 s1.

b) Calcular la longitud promedio de los unívocamente decodificables. (a, b, c, e)

Código	Longitudes	L (bits/símbolo)	$\eta = H/L$ ($H = 2$)
a	3,3,3,3,3,3	3	$2/3 = 66.67 \%$
b	1,2,3,4,5,6	$\frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + (3+4+5+6)/16 = 0.5+0.5+1.125 = \mathbf{2.125}$	$2/2.125 = 94.12 \%$
c	1,2,3,4,5,6	idéntica a b = 2.125	94.12 %
e	1,2,4,4,4,4	$\frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + 4 \cdot (4/16) = 0.5+0.5+1 = \mathbf{2}$	100 % (compacto: L = H)

El mejor es **e** ($L = H = 2$): las probabilidades son de la forma $2^{(-l_i)}$, de modo que existe un código 100 % eficiente.

c) En los códigos que presenten problemas, indicar qué falla y proponer una modificación mínima.

Código	Qué falla	Modificación mínima
b	No es instantáneo (cada palabra es prefijo de la siguiente).	Escribir cada palabra espejada (invertir el orden de sus bits) → 0, 10, 110, 1110, 11110, 111110 = código c. No cambian las longitudes ni L (2.125); el 0 pasa a ser terminador en lugar de inicio.
d	No es UD: $\Sigma 2^{-l} = 17/16 > 1$; hay prefijos (10 $\subset$ 1011, 110 $\subset$ 1101) y ambigüedad.	Alargar dos palabras: s3: 110 → 1100 y s5: 1011 → 1111 . Queda { 0, 10, 1100, 1110, 1111, 1101 }: sin prefijos, Kraft = 1, L = 2 = H (es el código e).
f	No es UD: $\Sigma 2^{-l} = 9/8 > 1$ (el árbol ya está lleno con 0 y las cuatro hojas 1xx) y 0 es prefijo de 001.	Reemplazar 111 y 001 por 1110 y 1111 : { 0, 100, 101, 110, 1110, 1111 } → Kraft = $1/2 + 3/8 + 2/16 = 1$, instantáneo, L = 2.125. Para que además sea compacto habría que reasignar longitudes como en e (1,2,4,4,4,4 → L = 2).

Observación: a y c no «fallan», pero no son óptimos (a desperdicia 2 de las 8 palabras de 3 bits y $L = 3 > H$; c tiene $L = 2.125 > 2$).

Ejercicio 2 — Longitudes válidas en base ternaria

Long. palabra →	1	2	3	4	5
Código A	2	1	2	1	4
Código B	2	2	2	3	1
Código C	1	4	6	0	0
Código D	2	2	2	2	3

Para que un conjunto de longitudes admita un código instantáneo (o UD) en base r, la condición de Kraft-McMillan generalizada es:

$$\sum_l n_l \cdot r^{-l} \leq 1$$

donde n_l es la cantidad de palabras de longitud l . Acá $r = 3$ (alfabeto ternario $\{0,1,2\}$) y l va de 1 a 5, así que:

$$\sum_{l=1}^5 n_l \cdot 3^{-l} = n_1 \cdot 3^{-1} + n_2 \cdot 3^{-2} + n_3 \cdot 3^{-3} + n_4 \cdot 3^{-4} + n_5 \cdot 3^{-5} \leq 1$$

Multiplicando toda la desigualdad por $3^5 = 243$ (el denominador común) solo para trabajar con números enteros.

$$81n_1 + 27n_2 + 9n_3 + 3n_4 + n_5 \leq 243$$

a) ¿Cuál de los conjuntos de longitudes de la siguiente tabla es válida para un código unívocamente decodificable con base $X\{0,1,2\}$?

Código	$n_1 \dots n_5$	palabras	$81n_1 + 27n_2 + 9n_3 + 3n_4 + n_5$	$\Sigma 3^{-l}$	Resultado
A	2, 1, 2, 1, 4	10	$2 \cdot 81 + 1 \cdot 27 + 2 \cdot 9 + 1 \cdot 3 + 4 \cdot 1 =$ 214	$214/243 =$ 0.8807	✓ válido
B	2, 2, 2, 3, 1	10	$2 \cdot 81 + 2 \cdot 27 + 2 \cdot 9 + 3 \cdot 3 + 1 \cdot 1 =$ 244	$244/243 =$ 1.0041	✗ no válido
C	1, 4, 6, 0, 0	11	$1 \cdot 81 + 4 \cdot 27 + 6 \cdot 9 =$ 243	$243/243 =$ 1.0000	✓ válido
D	2, 2, 2, 2, 3	11	$2 \cdot 81 + 2 \cdot 27 + 2 \cdot 9 + 2 \cdot 3 + 3 \cdot 1 =$ 243	$243/243 =$ 1.0000	✓ válido

Respuesta: son válidos **A** ($214/243 < 1$), **C** ($= 1$) y **D** ($= 1$). **B no** ($244/243 > 1$): por McMillan no existe ningún código unívocamente decodificable con esas longitudes, y menos uno instantáneo.

b) Construir un código instantáneo con cada conjunto de longitudes válidas (método de los apuntes:
 $n_i \leq r^{l_i} - \sum_{i < l} n_i r^{(l_i - i)}$)

Se toman las palabras más cortas primero; en cada nivel se cuentan los prefijos libres y se los extiende con los 3 símbolos.

Código A — $n = (2, 1, 2, 1, 4)$

Nivel l	Palabras disponibles (3·libres del nivel anterior) <i>r=3</i>	Se usan n_i	Quedan libres
1	3	2	1
2	3 <i>3×1</i>	1	2
3	6 <i>3×2</i>	2	4
4	12 <i>3×4</i>	1	11
5	33 <i>3×11</i>	4	29

[0,1,2]

palabra 1 = 0

palabra 2= 1

palabra 3 = 2

n_1 = usa 2 palabras de 3 ... 0 1 -> sobra 2

n_2 = usa 1 palabra de 3 ... 0 -> pero se utiliza el sobrante del nivel anterior que es el 2: 20 -> sobra 21, 22

n_3 = usa 2 palabras de 6 ... 0 1 -> 210, 211 -> sobra 212, 220, 221, 222

n_4 = usa 1 palabra de 12 ... 0 -> 2120 -> sobra 2121 ,2122, 2200,2201,2202,... y asi me faltan 11 palabras mas pero no las utilizamos a esos codigos es solamente para visualizar el proceso

n_5 = usa 4 palabras de 33... 0 1 2 y 0 para la última palabra -> 21210, 21211, 21212, 21220

Código obtenido (ordenado por longitud): (0, 1), (20), (210, 211), (2120), 21210, 21211, 21212, 21220...

Código C — n = (1, 4, 6, 0, 0)

Nivel I	Palabras disponibles (3·libres del nivel anterior)	Se usan n□	Quedan libres
1	3	1	2
2	6	4	2
3	6	6	0
4	0	0	0
5	0	0	0

Código obtenido (ordenado por longitud): 0, 10, 11, 12, 20, 210, 211, 212, 220, 221, 222

Código D — n = (2, 2, 2, 2, 3)

Nivel I	Palabras disponibles (3·libres del nivel anterior)	Se usan n□	Quedan libres
1	3	2	1
2	3	2	1
3	3	2	1

4	3	2	1
5	3	3	0

Código obtenido (ordenado por longitud): 0, 1, 20, 21, 220, 221, 2220, 2221, 22220, 22221, 22222

Verificación: ninguna palabra es prefijo de otra en A, C ni D; en A quedan 29 hojas de nivel 5 sin usar ($29/243 = 1 - 214/243$) y en C y D el árbol queda completo.

Ejercicio 3 — Demostrar que el código coma cumple Kraft

Definición. Para una fuente de q símbolos, el código coma binario asigna

$s_i \mapsto [1 \dots 1 \text{ (i - 1 unos)}] 0 \text{ (i = 1, \dots, q), } l_i = i,$

por ejemplo para $q = 4$: 0, 10, 110, 1110. El 0 es la «coma» que termina cada palabra.

Demostración. Hay que probar $\sum_{i=1}^q 2^{-l_i} \leq 1$. Con $l_i = i$ es una suma geométrica de razón $1/2$:

$$\sum_{i=1}^q 2^{-i} = (1/2)(1 - 2^{-(q)}) / (1/2) = 1 - 2^{-(q)} < 1.$$

Por inducción: para $q = 1$, $1/2 = 1 - 1/2$. Si vale para q , entonces $\sum_{i=1}^{q+1} 2^{-i} = (1 - 2^{-(q)}) + 2^{-(q+1)} = 1 - 2^{-(q+1)}$. ■

Generalización a base r (una sola palabra por longitud, la coma es el símbolo 0): $\sum_{i=1}^q r^{-i} = (1 - r^{-(q)})/(r - 1) \leq 1$ para todo $r \geq 2$.

Comentarios.

- Si la última palabra no lleva coma (0, 10, 110, ..., $1^{q-2}0$, 1^{q-1}) las dos últimas tienen longitud $q - 1$ y la suma vale exactamente $\sum_{i=1}^{q-1} 2^{-i} + 2^{-(q-1)} = 1$: el código queda *completo*.
- Coherencia con la teoría: el código coma es instantáneo (el 0 marca el fin de palabra) y todo código instantáneo debe cumplir Kraft; aquí la cumple con holgura $2^{-(q)}$, que es la hoja no usada 1^q del árbol.
- Ejemplo numérico: $q = 6$ (código c del ej. 1): $1/2 + 1/4 + \dots + 1/64 = 63/64$.

Ejercicio 4 — Sea un código trinario. Codificar los diez dígitos decimales con longitudes 1, 2, 2, 2, 2, 2, 3, 3, 3, 3. Si no se pudiera ¿hasta cuantos dígitos podría codificar?, Realice el árbol de codificación.

Longitudes pedidas: 1, 2, 2, 2, 2, 2, 3, 3, 3, 3 $\rightarrow n_1 = 1, n_2 = 5, n_3 = 4$, base $r = 3$.

$$\sum_l n_l \cdot r^{-l} = n_1 \cdot 3^{-1} + n_2 \cdot 3^{-2} + n_3 \cdot 3^{-3}$$

$$= \underbrace{1}_{n_1} \cdot \underbrace{\frac{1}{3}}_{3^{-1}} + \underbrace{5}_{n_2} \cdot \underbrace{\frac{1}{9}}_{3^{-2}} + \underbrace{4}_{n_3} \cdot \underbrace{\frac{1}{27}}_{3^{-3}}$$

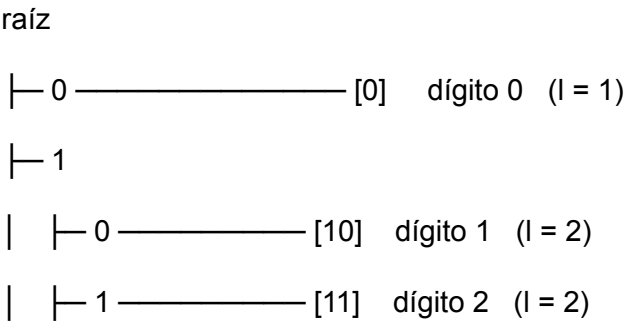
$$= \frac{1}{3} + \frac{5}{9} + \frac{4}{27}$$

Kraft: $1/3 + 5/9 + 4/27 = (9 + 15 + 4)/27 = 28/27 > 1 \Rightarrow$ **no es posible** un código instantáneo (ni UD) con esas longitudes.

Nivel	Disponibles (3 × libres del nivel anterior)	Se piden	Quedan libres
1	3	1	2
2	6	5	1
3	3	4	✗ falta 1 hoja (se piden 4, hay 3)

¿Hasta cuántos dígitos se puede codificar? Tras usar 1 palabra de longitud 1 y 5 de longitud 2 queda 1 prefijo libre de longitud 2, que se extiende en 3 hojas de longitud 3 (cada una pesa 1/27; la suma parcial ya es 24/27). Por lo tanto solo caben **3** palabras de longitud 3 y se pueden codificar **9 dígitos** (1 + 5 + 3; Kraft = 27/27 = 1, árbol completo). El décimo dígito queda sin código.

Árbol de codificación (base 3, alfabeto {0,1,2}; se asignan los dígitos 0–8 en orden de longitud):



| └ 2 ————— [12] dígito 3 (l = 2)

└ 2

├ 0 ————— [20] dígito 4 (l = 2)

├ 1 ————— [21] dígito 5 (l = 2)

└ 2

├ 0 ————— [220] dígito 6 (l = 3)

├ 1 ————— [221] dígito 7 (l = 3)

└ 2 ————— [222] dígito 8 (l = 3) → no queda ninguna hoja libre para el dígito 9

Código: 0→0, 1→10, 2→11, 3→12, 4→20, 5→21, 6→220, 7→221, 8→222. Árbol completo: $\sum 3^{(-l_i)} = 1$.

Cómo codificar los 10 dígitos: hay que alargar alguna palabra. Por ejemplo con longitudes 1,2,2,2,2,2,3,3,4,4 (Kraft = $(27 + 45 + 6 + 2)/81 = 80/81$): 0, 10, 11, 12, 20, 21, 220, 221, 2220, 2221.

Ejercicio 5 — Entropía y codificación estadística: Sea la prob. de la fuente: A= 0.30, B=0.25, C=0.18, D=0.12, E=0.08, F=0.07

a) Calcular la entropía H(x)

Símbolo	p	$\log_2(1/p)$	$p \cdot \log_2(1/p)$
A	30/100=0.30	1.7370	0.5211
B	25/100 = 0.25	2.0000	0.5000
C	18/100 = 0.18	2.4739	0.4453

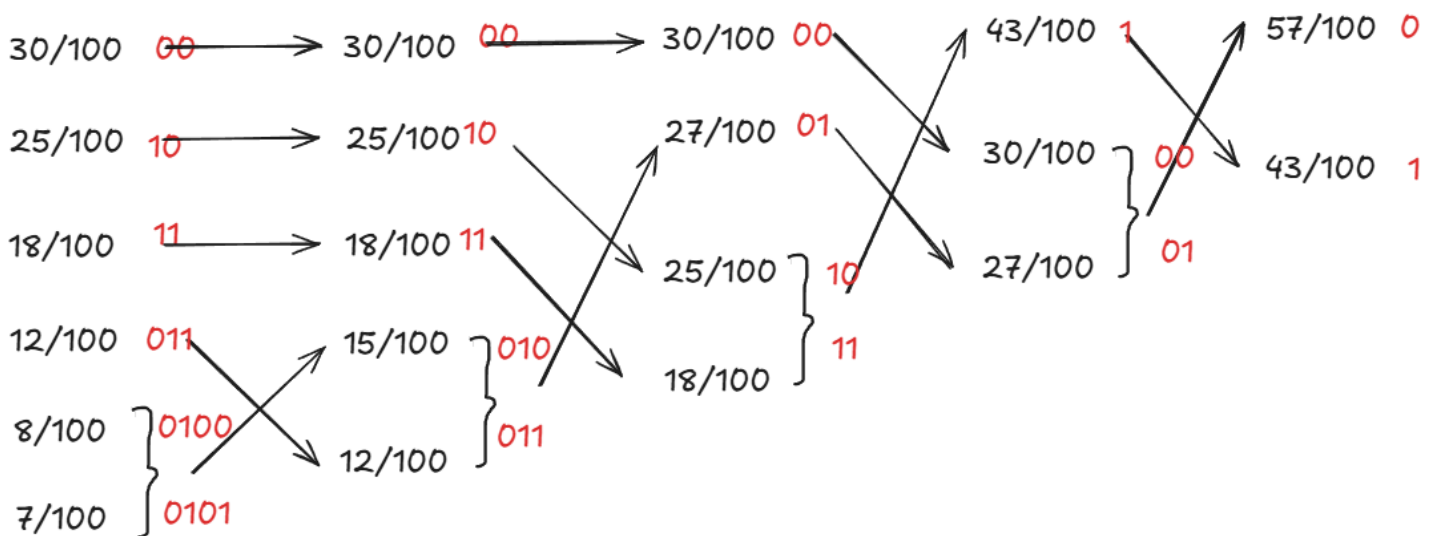
D	$12/100 = 0.12$	3.0589	0.3671
E	$8/100 = 0.08$	3.6439	0.2915
F	$7/100 = 0.07$	3.8365	0.2686

$$H(X) = \sum p_i \log_2 1/p_i = 2.3935 \text{ bits/símbolo}$$

b) Construir los códigos de Shannon, Fano y Huffman

c) Calcular para cada método la longitud promedio L , la eficiencia $\eta = H(X)/L$ y la

HUFFMAN ESTÁTICO



$$L_{\text{huffman}} = \frac{30 \times 2 + 25 \times 2 + 18 \times 2 + 12 \times 3 + 8 \times 4 + 7 \times 4}{100} = 2.42 \text{ bits/símbolo}$$

$$\eta = H(X)/L = 2.3935 \text{ bits/símbolo} / 2.42 \text{ bits/símbolo} = 0.9891 = 98\%$$

$$R = 1 - \eta = 1 - 0.9891 = 0.0109 = 1.09\%$$

SHANNON

Longitud Individual

$1.73 \leq A \leq 2.73$	$\log_2 30/100 \leq A \leq \log_2 30/100 + 1 \rightarrow$	2
$2 \leq B \leq 3$	$\log_2 25/100 \leq B \leq \log_2 25/100 + 1 \rightarrow$	2
$2.47 \leq C \leq 3.47$	$\log_2 18/100 \leq C \leq \log_2 18/100 + 1 \rightarrow$	3
$3.05 \leq D \leq 4.05$	$\log_2 12/100 \leq D \leq \log_2 12/100 + 1 \rightarrow$	4
$3.64 \leq E \leq 4.64$	$\log_2 8/100 \leq E \leq \log_2 8/100 + 1 \rightarrow$	4
$3.83 \leq F \leq 4.83$	$\log_2 7/100 \leq F \leq \log_2 7/100 + 1 \rightarrow$	4

Simbolo	p_i	FA	Dec	Bin	Long	Cod
A	30/100	0	0	0	2	0
B	25/100	30/100	0.3	0.01	2	01
C	18/100	55/100	0.55	0.100	3	100
D	12/100	73/100	0.73	0.1011	4	1011
E	8/100	85/100	0.85	0.1101	4	1101
F	7/100	93/100	0.93	0.1110	4	1110

$$L_{\text{shannon}} = \frac{30 \times 2 + 25 \times 2 + 18 \times 3 + 12 \times 4 + 8 \times 4 + 7 \times 4}{100} = 2.72 \text{ bits/simbolo}$$

$$n = H(X)/L = 2.3935 \text{ bits/simbolo} / 2.72 \text{ bits/simbolo} = 0.8799 = 88\%$$

$$R = 1 - n = 1 - 0.8799 = 0.1201 = 12.01\%$$

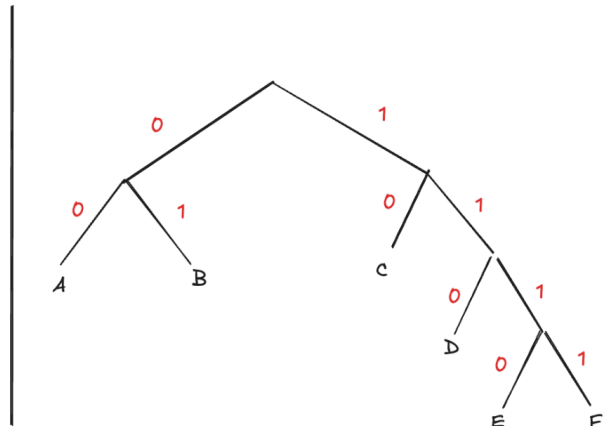
FANO

A	30/100	00		
B	25/100	01	0.55	0
C	18/100	10	0.45	1
D	12/100	110		
E	8/100	1110		
F	7/100	1111		

$$L_{\text{fanon}} = \frac{30 \times 2 + 25 \times 2 + 18 \times 2 + 12 \times 3 + 8 \times 4 + 7 \times 4}{100} = 2.42 \text{ bits/simbolo}$$

$$n = H(X)/L = 2.3935 \text{ bits/simbolo} / 2.42 \text{ bits/simbolo} = 0.9890 = 98\%$$

$$R = 1 - n = 1 - 0.9890 = 0.0109 = 1.09\%$$



d) Comparar los resultados y justificar cuál se aproxima mejor al límite teórico.

- $L_{\text{Huffman}} = L_{\text{Fano}} = 2.42 < L_{\text{Shannon}} = 2.72$.
- **Huffman** es el más cercano al límite (óptimo): dista $2.42 - 2.3935 = 0.0265$ bits de H ($\eta = 98.91\%$). Fano empata aquí por tener el mismo perfil de longitudes.
- **Shannon** queda a 0.3265 bits de H ($\eta = 88.00\%$): el redondeo hacia arriba desperdicia bits y su árbol queda incompleto ($\sum 2^{-l_i} = 0.8125 < 1$).
- Ninguno alcanza H porque las p_i no son de la forma 2^{-l_i} .

Ejercicio 6 — Método de diccionario: LZ y LZW. Trabajar con la cadena:

ERRECONERRECIGARROERRECONERREBARRIL

a) Aplicar LZ utilizando una ventana de búsqueda de 6 símbolos y una ventana de anticipación de 4 símbolos. Mostrar las tuplas generadas y reconstruir la cadena

PASO	BUFFER DE BÚSQUEDA (máx. 6)	BUFFER DE ANTICIPACIÓN (máx. 4)	COINCIDENCIA HACIA ATRÁS	SALIDA
1	(vacío)	ERRE	No hay, sig. E	(0,0,'E')
2	E	RREC	No hay, sig. R	(0,0,'R')
3	ER	RECO	Hay coinc. "R", a 1 pos, long 1	(1,1,'E')
4	ERRE	CONE	No hay, sig. C	(0,0,'C')
5	ERREC	ONER	No hay, sig. O	(0,0,'O')
6	ERRECO	NERR	No hay, sig. N	(0,0,'N')
7	RRECON	ERRE	Hay coinc. "E", a 4 pos, long 1	(4,1,'R')
8	ECONER	RECI	Hay coinc. "R", a 1 pos, long 1	(1,1,'E')
9	ONERRE	CIGA	No hay, sig. C	(0,0,'C')

10	NERREC	IGAR	No hay, sig. I	(0,0,'I')
11	ERRECI	GARR	No hay, sig. G	(0,0,'G')
12	RRECIG	ARRO	No hay, sig. A	(0,0,'A')
13	RECIGA	RROE	Hay coinc. "R", a 6 pos, long 1	(6,1,'R')
14	CIGARR	OERR	No hay, sig. O	(0,0,'O')
15	IGARRO	ERRE	No hay, sig. E	(0,0,'E')
16	GARROE	RREC	Hay coinc. "RR", a 4 pos, long 2	(4,2,'E')
17	ROERRE	CONE	No hay, sig. C	(0,0,'C')
18	OERREC	ONER	Hay coinc. "O", a 6 pos, long 1	(6,1,'N')
19	RRECON	ERRE	Hay coinc. "E", a 4 pos, long 1	(4,1,'R')
20	ECONER	REBA	Hay coinc. "R", a 1 pos, long 1	(1,1,'E')
21	ONERRE	BARR	No hay, sig. B	(0,0,'B')

22	NERREB	ARRI	No hay, sig. A	(0,0,'A')
23	ERREBA	RRIL	Hay coinc. "RR", a 5 pos, long 2	(5,2,'I')
24	EBARRI	L	No hay, sig. L	(0,0,'L')

Salida final (24 marcas para 35 símbolos):
(0,0,'E') (0,0,'R') (1,1,'E') (0,0,'C') (0,0,'O') (0,0,'N') (4,1,'R') (1,1,'E') (0,0,'C') (0,0,'I') (0,0,'G') (0,0,'A') (6,1,'R') (0,0,'O') (0,0,'E') (4,2,'E') (0,0,'C') (6,1,'N') (4,1,'R') (1,1,'E') (0,0,'B') (0,0,'A') (5,2,'I') (0,0,'L')

b) Aplicar LZW usando como diccionario inicial los símbolos distintos de la cadena. Mostrar la evolución del diccionario, la secuencia de códigos emitidos y la descompresión.

Lempel-Ziv-Welch (LZW)

Mensaje: ERRECONERRECIGARROERRECONERREBARRIL (35 símbolos)

Diccionario inicial (códigos ASCII de los símbolos que aparecen)

Símbolo	Código
A	65
B	66
C	67
E	69
G	71

I	73
L	76
N	78
O	79
R	82

Las nuevas frases se guardan a partir del índice **256**.

Tabla completa

Paso (lectura)	Cadena Acumulada (I)	Evaluación (I+x)	Salida (Código Emitido)	Nueva entrada añadida al dicc.
1 (lee 'E')	E	Lee R, "ER" no está en el dicc	69 índice de E	256 = ER
2 (lee 'R')	R	Lee R, "RR" no está en el dicc	82 índice de R	257 = RR
3 (lee 'R')	R	Lee E, "RE" no está en el dicc	82 índice de R	258 = RE
4 (lee 'E')	E	Lee C, "EC" no está en el dicc	69 índice de E	259 = EC

5 (lee 'C')	C	Lee O, "CO" no está en el dicc	67 índice de C	260 = CO
6 (lee 'O')	O	Lee N, "ON" no está en el dicc	79 índice de O	261 = ON
7 (lee 'N')	N	Lee E, "NE" no está en el dicc	78 índice de N	262 = NE
8 (lee 'E')	E	Lee R, "ER" sí está en dicc (256)	–	–
9 (lee 'R')	ER	Lee R, "ERR" no está en el dicc	256 índice de ER	263 = ERR
10 (lee 'R')	R	Lee E, "RE" sí está en dicc (258)	–	–
11 (lee 'E')	RE	Lee C, "REC" no está en el dicc	258 índice de RE	264 = REC
12 (lee 'C')	C	Lee I, "CI" no está en el dicc	67 índice de C	265 = CI
13 (lee 'I')	I	Lee G, "IG" no está en el dicc	73 índice de I	266 = IG
14 (lee 'G')	G	Lee A, "GA" no está en el dicc	71 índice de G	267 = GA

15 (lee 'A')	A	Lee R, "AR" no está en el dicc	65 índice de A	268 = AR
16 (lee 'R')	R	Lee R, "RR" sí está en dicc (257)	–	–
17 (lee 'R')	RR	Lee O, "RRO" no está en el dicc	257 índice de RR	269 = RRO
18 (lee 'O')	O	Lee E, "OE" no está en el dicc	79 índice de O	270 = OE
19 (lee 'E')	E	Lee R, "ER" sí está en dicc (256)	–	–
20 (lee 'R')	ER	Lee R, "ERR" sí está en dicc (263)	–	–
21 (lee 'R')	ERR	Lee E, "ERRE" no está en el dicc	263 índice de ERR	271 = ERRE
22 (lee 'E')	E	Lee C, "EC" sí está en dicc (259)	–	–
23 (lee 'C')	EC	Lee O, "ECO" no está en el dicc	259 índice de EC	272 = ECO
24 (lee 'O')	O	Lee N, "ON" sí está en dicc (261)	–	–

25 (lee 'N')	ON	Lee E, "ONE" no está en el dicc	261 índice de ON	273 = ONE
26 (lee 'E')	E	Lee R, "ER" sí está en dicc (256)	–	–
27 (lee 'R')	ER	Lee R, "ERR" sí está en dicc (263)	–	–
28 (lee 'R')	ERR	Lee E, "ERRE" sí está en dicc (271)	–	–
29 (lee 'E')	ERRE	Lee B, "ERREB" no está en el dicc	271 índice de ERRE	274 = ERREB
30 (lee 'B')	B	Lee A, "BA" no está en el dicc	66 índice de B	275 = BA
31 (lee 'A')	A	Lee R, "AR" sí está en dicc (268)	–	–
32 (lee 'R')	AR	Lee R, "ARR" no está en el dicc	268 índice de AR	276 = ARR
33 (lee 'R')	R	Lee I, "RI" no está en el dicc	82 índice de R	277 = RI
34 (lee 'I')	I	Lee L, "IL" no está en el dicc	73 índice de I	278 = IL
35 (lee 'L')	L	fin de cadena	76 índice de L	EOF

Salida final

24 códigos (en vez de 35 símbolos sueltos):

69, 82, 82, 69, 67, 79, 78, 256, 258, 67, 73, 71, 65, 257, 79, 263, 259, 261, 271, 66, 268, 82, 73, 76

Resultado esperado. Diccionario inicial (índices desde 0): A=0, B=1, C=2, E=3, G=4, I=5, L=6, N=7, O=8, R=9.

- Códigos emitidos (24): 3 9 9 3 2 8 7 10 12 2 5 4 0 11 8 17 13 15 25 1 22 9 5 6
- Diccionario al final: 33 entradas (nuevas: 10 a 32); últimas: 30=ARR, 31=RI, 32=IL.
- La descompresión reproduce la cadena original; no aparece el caso especial KwKwK.
- (Si la cátedra numera desde 1, sumar 1 a todos los códigos.)

c) Comparar qué tipo de repetición aprovecha cada método.

	LZ77 (ventana 6/4)	LZW
Repetición que explota	Subcadenas recientes : solo se ve lo que está a ≤ 6 posiciones atrás; las coincidencias pueden solaparse con la anticipación (rachas tipo RRRR).	Frases repetidas a cualquier distancia : el diccionario acumula todo lo visto y va alargando las frases de a un símbolo por vez.
Qué se transmite	Punteros (d, l, c); el «diccionario» es la propia ventana.	Solo índices; el diccionario se reconstruye en el decodificador.
En esta cadena	Solo aprovecha coincidencias locales de 1 o 2 símbolos (R, E, O, RR). No ve que ERRECON reaparece 18 posiciones más adelante, ni que ERRE se repite a distancias 7, 11 y 7: todas exceden la ventana de 6.	Aprende ER→RE→ERR→ERRE y luego reutiliza: los códigos 17 (ERR) y 25 (ERRE) reemplazan 3 y 4 símbolos.
Debilidad	Ventana chica ⇒ casi todo son literales (15 de 24 tokens).	Necesita ver una frase varias veces para que crezca; en textos cortos se adapta lento.

Tamaño aproximado (supuestos: 4 bits por símbolo del alfabeto de 10; tokens LZ77 de 3+2+4 = 9 bits; códigos LZW de 6 bits porque el diccionario llega a 33 entradas): LZ77 = 24×9 = **216 bits**, LZW = 24×6 = **144 bits**; original = 140 bits (bloque de 4 bits) o 280 bits (ASCII). Frente a ASCII: LZ77 ahorra 22.9 % y LZW 48.6

%; frente al bloque de 4 bits ninguno mejora, porque la cadena es demasiado corta (los métodos de diccionario rinden con textos largos y muy repetitivos).

Ejercicio 7 — Codificación aritmética: Sea la fuente con $p(A)=0,50$; $p(B)=0,30$; $p(C)=0,20$ y la secuencia: ABACA

Fuente: $p(A) = 0.50$, $p(B) = 0.30$, $p(C) = 0.20$; secuencia ABACA.

a) Codificar la secuencia mediante codificación aritmética. Mostrar los intervalos iniciales, las subdivisiones sucesivas y determinar el intervalo final. Elegir un valor perteneciente a dicho intervalo como representación de la secuencia.

Intervalos iniciales (acumuladas Q y R de cada símbolo):

Símbol o	p	Q (límite inferior)	R (límite superior)	Intervalo
A	0.50	0.0	0.5	[0.0, 0.5)
B	0.30	0.5	0.8	[0.5, 0.8)
C	0.20	0.8	1.0	[0.8, 1.0)

Subdivisiones sucesivas con $\Delta = V - U$, $U_{\text{nuevo}} = U + \Delta Q$, $V_{\text{nuevo}} = U + \Delta R$:

Paso / símbolo	Operación	U (inferior)	V (superior)	Δ
inicio	—	0.0	1.0	1.0
1: A	$U = 0 + 1 \cdot 0$ $V = 0 + 1 \cdot 0.5$	0	0.5	0.5

2: B	$U = 0 + 0.5 \cdot 0.5$ $V = 0 + 0.5 \cdot 0.8$	0.25	0.4	0.15
3: A	$U = 0.25 + 0.15 \cdot 0$ $V = 0.25 + 0.15 \cdot 0.5$	0.25	0.325	0.075
4: C	$U = 0.25 + 0.075 \cdot 0.8$ $V = 0.25 + 0.075 \cdot 1$	0.31	0.325	0.015
5: A	$U = 0.31 + 0.015 \cdot 0$ $V = 0.31 + 0.015 \cdot 0.5$	0.31	0.3175	0.0075

Intervalo final: $[0.31, 0.3175)$ de ancho $0.0075 = p(A)^3 p(B) p(C) = 0.5^3 \cdot 0.3 \cdot 0.2$ ✓ (información $-\log_2 0.0075 = 7.06$ bits).

Valor representante: se busca la fracción binaria más corta dentro del intervalo:

Bits	Menor múltiplo de $2^{-n} \geq U$	$z \in [0.31, 0.3175)$?
1	$1/2 = 0.5$	✗ fuera
2	$2/4 = 0.5$	✗ fuera
3	$3/8 = 0.375$	✗ fuera
4	$5/16 = 0.3125$	✓ está dentro
5	$10/32 = 0.3125$	✓ está dentro

$0.3125 = 5/16 = 0.0101_2$ pertenece al intervalo con solo 4 bits $\Rightarrow$ **código = 0101** (más la longitud $N = 5$ o un símbolo EOF para saber cuándo cortar).

(Nota: la cota garantizada es $\lceil \log_2 (1/P) \rceil + 1 = 9$ bits; aquí basta con 4 porque el intervalo contiene por casualidad una fracción diádica muy corta.)

b) A partir del valor obtenido, realizar el proceso de decodificación y verificar que se recupera la secuencia original.

El receptor tiene $val = 0.3125$ y $N = 5$. En cada paso ubica el intervalo que contiene a val y re-escala:
 $val_nuevo = (val - Q)/P$.

Pas o	val	Cae en	Símbolo	Re-escalado
1	0.3125 (5/16)	[0, 0.5)	A	$(0.3125 - 0) / 0.5 = 0.625$
2	0.625 (5/8)	[0.5, 0.8)	B	$(0.625 - 0.5) / 0.3 = 0.416667$
3	0.416667 (5/12)	[0, 0.5)	A	$(0.416667 - 0) / 0.5 = 0.833333$
4	0.833333 (5/6)	[0.8, 1)	C	$(0.833333 - 0.8) / 0.2 = 0.166667$
5	0.166667 (1/6)	[0, 0.5)	A	$(0.166667 - 0) / 0.5 = 0.333333$

Secuencia recuperada: **ABACA** = **ABACA**.

Práctico 3 — Fuentes de Markov

Ejercicio 1 — Dado el siguiente enunciado construir un conjunto de probabilidades condicionadas (con cuatro estados) en un proceso de Markov de primer orden, encontrar la matriz de transición y el grafo.

Secuencia: **AABABACABABACABBABAACAB | AC**

Estados. El alfabeto es {A, B, C} ($q = 3$); en un proceso de **primer orden** los estados son $q^m = 3^1 = 3$ (uno por símbolo anterior). La barra | separa el cuerpo de la secuencia de su cierre; se usa la secuencia completa: 25 símbolos $\rightarrow$ **24 transiciones** (con **AB|AC** aparecen las transiciones $B \rightarrow A$ y $A \rightarrow C$).

pos: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

sím: A A B A B A C A B A B A C A B B A B A A C A B A C

Frecuencias: A = 13, B = 8, C = 4 (total 25) $\Rightarrow p(A) = 13/25$, $p(B) = 8/25$, $p(C) = 4/25$.

Conteo de transiciones n_{ij} (fila = estado actual, columna = símbolo siguiente):

Desde $\rightarrow$ Hacia	A	B	C	Σ fila
A	2	7	4	13
B	7	1	0	8
C	3	0	0	3
Total	12	8	4	24

Conjunto de probabilidades condicionadas $p(s_i|s_j) = n_{ji}/\Sigma_j n_{ji}$:

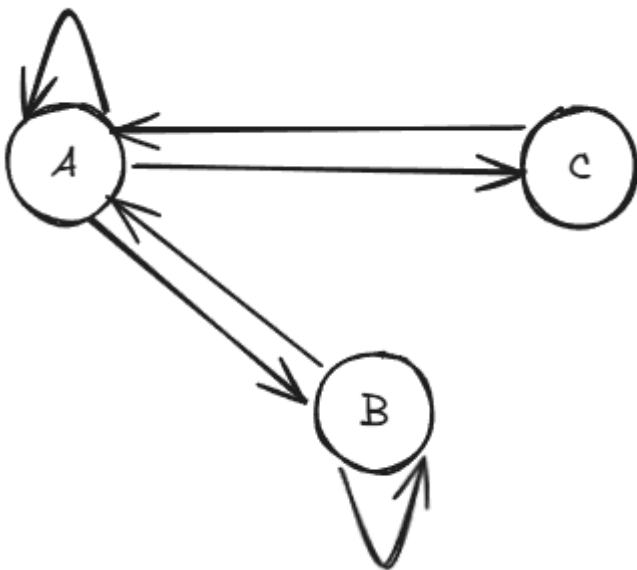
Estado	$p(A \cdot)$	$p(B \cdot)$	$p(C \cdot)$
$p(\cdot A)$	2/13	7/13	4/13
$p(\cdot B)$	7/8	1/8	0

$p(\cdot C)$	1	0	0
----------------	---	---	---

Matriz de transición (cada fila suma 1):

$2/13$	$7/13$	$4/13$
$7/8$	$1/8$	0
1	0	0

Grafo de estados:



Ejercicio 2) A partir del ejercicio n°1

a) Obtener la matriz estacionaria (distribución a la que tiende).

Método analítico: $\pi P = \pi$ con $\sum \pi_i = 1$:

$$\pi_A = (2/13)\pi_A + (7/8)\pi_B + \pi_C$$

$$\pi_B = (7/13)\pi_A + (1/8)\pi_B$$

$$\pi_C = (4/13)\pi_A$$

$$\pi_A + \pi_B + \pi_C = 1$$

De la 3.ª: $\pi_C = 4/13 \pi_A$. De la 2.ª: $7/8 \pi_B = 7/13 \pi_A \Rightarrow \pi_B = 8/13 \pi_A$. Normalizando: $\pi_A(1 + 8/13 + 4/13) = 25/13 \pi_A = 1$.

$$\pi = (13/25, 8/25, 4/25) = (0.52, 0.32, 0.16)$$

Coincide con las frecuencias relativas de la secuencia (la secuencia se cierra sobre sí misma: C solo va a A, y la primera letra es A). **Matriz estacionaria:** $\Pi = \lim P^n$ tiene las tres filas iguales a π .

b) Comprobar que las potencias sucesivas de la matriz de transición tienen menor variabilidad en los valores de sus componentes.

$$P^2 =$$

0.80251 0.15015 0.04734

0.24399 0.48678 0.26923

0.15385 0.53846 0.30769

$$P^3 =$$

0.30218 0.45089 0.24693

0.73270 0.19223 0.07507

0.80251 0.15015 0.04734

$$P^4 =$$

0.68795 0.21907 0.09298

0.35600 0.41856 0.22545

0.30218 0.45089 0.24693

$$P^8 =$$

0.57936 0.28433 0.13631

0.46204 0.35483 0.18313

0.44302 0.36626 0.19072

$P^{16} =$

0.52741 0.31554 0.15704

0.51276 0.32435 0.16289

0.51038 0.32578 0.16384

$P^{32} =$

0.52012 0.31993 0.15995

0.51989 0.32007 0.16005

0.51985 0.32009 0.16006

Variabilidad medida como amplitud de cada columna (máximo – mínimo entre las filas): tiende a 0 porque las filas de P^n se van igualando a π .

n	amplitud col. A	amplitud col. B	amplitud col. C	máx.
1	0.84615	0.53846	0.30769	0.84615
2	0.64867	0.38831	0.26036	0.64867
3	0.50033	0.30074	0.19959	0.50033
4	0.38577	0.23182	0.15395	0.38577
5	0.29744	0.17874	0.11870	0.29744

6	0.22934	0.13782	0.09152	0.22934
8	0.13634	0.08193	0.05441	0.13634
12	0.04819	0.02896	0.01923	0.04819
16	0.01703	0.01023	0.00680	0.01703
32	0.00027	0.00016	0.00011	0.00027

La amplitud decrece **monótonamente** en cada columna ($0.846 \rightarrow 0.649 \rightarrow 0.500 \rightarrow 0.386 \rightarrow \dots \rightarrow 0.0003$), a razón aproximada $|\lambda_2|^n$ con $\lambda_2 = -0.771$ (autovalores de P: 1, -0.771, 0.050). Por eso $P^{32} \approx \Pi$ con error $< 10^{-3}$.

c) Comprobar que la distribución estacionaria es la misma para el cuadrado de la matriz de transición.

$$P^2 = \begin{vmatrix} 1085/1352 & 203/1352 & 8/169 \\ 203/832 & 405/832 & 7/26 \\ 2/13 & 7/13 & 4/13 \end{vmatrix}$$

d) ¿Es un proceso ergódico?

Sí. Ya que se cumple:

(i) *Irreducible*: $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow A$, $C \rightarrow A$, así que desde cualquier estado se llega a cualquiera.

(ii) *Aperiódico*: $A \rightarrow A$ y $B \rightarrow B$ son lazos (ciclos de longitud 1), por lo que el mcd de las longitudes de ciclo es 1.

Equivalentemente, P^2 ya tiene **todos los elementos positivos** (el proceso es *regular*). Por lo tanto existe una única distribución estacionaria, independiente del estado inicial, y $P^n \rightarrow \Pi$ (lo que se verificó numéricamente en (b)).

Ejercicio 3) Demostrar que las filas de la matriz de transición son linealmente dependientes en un proceso de Markov.

Demostración. Sea $P = (p_{ij})$ de $q \times q$ con $p_{ij} \geq 0$ y $\sum_j p_{ij} = 1$ para todo i (cada fila es una distribución). El sistema estacionario $\pi P = \pi$ se escribe, ecuación por ecuación (una por estado j):

$$E_j: \sum_i \pi_i p_{ij} - \pi_j = 0 \quad (j = 1, \dots, q),$$

es decir, $(P^T - I) \pi^T = 0$. La fila j de $P^T - I$ es $(p_{1j} - 1, \dots, p_{jj} - 1, \dots, p_{qj})$. Sumando las q ecuaciones, el coeficiente de cada π_i es

$$\sum_j p_{ij} - 1 = 1 - 1 = 0.$$

Entonces $E_1 + E_2 + \dots + E_q = 0$: existe una combinación lineal **no trivial** (todos los coeficientes iguales a 1) de las filas de $P^T - I$ que da la fila nula. Por lo tanto las filas son linealmente dependientes, $\text{rango}(P^T - I) \leq q - 1$ y $\det(P - I) = \det(P^T - I) = 0$. ■

Forma equivalente: $P \cdot \mathbf{1} = \mathbf{1}$ (las filas suman 1) $\Rightarrow (P - I)\mathbf{1} = 0 \Rightarrow$ las **columnas** de $P - I$ son dependientes $\Rightarrow P - I$ es singular $\Rightarrow 1$ es autovalor de P .

Consecuencia práctica. Una de las q ecuaciones es redundante: el sistema homogéneo tiene infinitas soluciones (una recta) y por eso hay que agregar la condición de normalización $\sum \pi_i = 1$ para obtener la distribución estacionaria única (como en el ej. 2.a).

Comprobación con la matriz del ej. 1:

$$P^T - I = \begin{vmatrix} -11/13 & 7/8 & 1 \\ 7/13 & -7/8 & 0 \\ 4/13 & 0 & -1 \end{vmatrix} \quad \text{suma de filas} = (0, 0, 0).$$

Ejercicio 4) Analizar el siguiente ejemplo y responder los enunciados:

$$S = (s_1, s_2, s_3, s_4) \quad p(s_i | (s_{j1}, s_{j2}, s_{j3}))$$

a) ¿De qué orden es el sistema de Markov? El símbolo depende de los 3 anteriores: proceso de Markov de orden $m = 3$.

b) ¿En cuántos estados puede encontrarse el sistema? Un estado es la terna de los últimos m símbolos: $q^m = 4^3 = 64$ estados $(s_{j1} s_{j2} s_{j3})$. La matriz de transición es de 64×64 , pero muy rara: desde $(x_1 x_2 x_3)$ solo se puede pasar a $(x_2 x_3 x_4)$, es decir a 4 estados como máximo, con probabilidad $p(x_4 | x_1 x_2 x_3)$.

c) Supuestas conocidas todas las probabilidades condicionales ¿cuál será la forma del sistema de ecuaciones para encontrar las $p(s_i)$? Sea $P(x_1 x_2 x_3)$ la probabilidad estacionaria del estado. Para cada estado destino $(x_2 x_3 x_4)$:

$$P(x_2 x_3 x_4) = \sum_{x_1=1..4} P(x_1 x_2 x_3) p(x_4 | x_1 x_2 x_3) \quad (64 \text{ ecuaciones, 4 términos cada una})$$

más la normalización $\sum P(x_1 x_2 x_3) = 1$. Matricialmente $\pi = \pi P$ con $\pi \in \mathbb{R}^{64}$; como el rango de $P - I$ es a lo sumo 63 (ej. 3), una ecuación es redundante y se reemplaza por la normalización (**64 incógnitas, 63 ecuaciones independientes + 1 de normalización**).

Una vez resuelto, las probabilidades de los símbolos individuales son las marginales:

$$p(s_i) = \sum_{(x_2, x_3)} P(s_i x_2 x_3).$$

(No se puede plantear un sistema de solo 4 ecuaciones en los $p(s_i)$, cómo en primer orden, porque $p(s_i)$ depende de la distribución conjunta de las ternas.)

Ejercicio 5) Supongamos una máquina con dos estados, anda=1, no anda=2. Estudiamos el proceso día a día llegando a comprobar que:

- i. si anda hoy, hay un 75% de chances que ande mañana
- ii. si no anda hoy, la arreglan mañana

a) Encontrar la Matriz de transición

$$P = \begin{vmatrix} 3/4 & 1/4 \\ 1 & 0 \end{vmatrix}$$

b) Calcular hasta la quinta potencia de la matriz de transición y analizar los resultados

Potencia	Fila 1 (exacta)	Fila 2 (exacta)	Fila 1 (decimal)	Fila 2 (decimal)
P^1	3/4, 1/4	1, 0	0.750000, 0.250000	1.000000, 0.000000
P^2	13/16, 3/16	3/4, 1/4	0.812500, 0.187500	0.750000, 0.250000
P^3	51/64, 13/64	13/16, 3/16	0.796875, 0.203125	0.812500, 0.187500
P^4	205/256, 51/256	51/64, 13/64	0.800781, 0.199219	0.796875, 0.203125
P^5	819/1024, 205/1024	205/256, 51/256	0.799805, 0.200195	0.800781, 0.199219

Análisis. Las dos filas convergen a la misma pareja (0.8, 0.2): la memoria del estado inicial se pierde. Las diferencias respecto del límite se multiplican por -1/4 en cada paso (el segundo autovalor de P es $\lambda_2 = -1/4$), por eso los valores **oscilan** alrededor de 0.8 —0.75, 0.8125, 0.7969, 0.8008, 0.7998— con error que se divide por 4 cada vez. En forma cerrada:

$$P^n = \Pi + (-1/4)^n(I - \Pi), P^{n+1} = 0.8 + 0.2(-1/4)^n$$

(verificación: $n = 2 \Rightarrow 0.8 + 0.2/16 = 0.8125$ ✓). Ya con P^5 los elementos difieren de π en menos de 10^{-3} .

c) Encontrar la matriz de equilibrio o estacionaria

$$\pi_1 = 3/4 \pi_1 + \pi_2, \pi_1 + \pi_2 = 1 \Rightarrow 1/4 \pi_1 = \pi_2 \Rightarrow \pi_1 = 0.8, \pi_2 = 0.2.$$

$$\Pi = \begin{bmatrix} 0.8 & 0.2 \end{bmatrix}$$

$$\begin{bmatrix} 0.8 & 0.2 \end{bmatrix}$$

A largo plazo la máquina anda el 80 % de los días y está rota el 20 %. El proceso es ergódico (irreducible y aperiódico por el lazo $1 \rightarrow 1$).

Ejercicio 6) Dado un proceso de Markov con tres estados a,b,c; con las siguientes probabilidades: $p(a/a)=3/4$; $p(c/c)=p(b/b)=1/2$; $p(b/a)=p(c/a)=1/8$; $p(a/b)=p(c/b)=p(a/c)=p(b/c)=1/4$ comprobar que la longitud de Markov para un código compacto $LM = 11/8 < 12/8 = L$

Probabilidades dadas $p(j|i)$ (fila = estado actual i):

$$P = \begin{bmatrix} 3/4 & 1/8 & 1/8 \end{bmatrix}$$

$$\begin{bmatrix} 1/4 & 1/2 & 1/4 \end{bmatrix}$$

$$\begin{bmatrix} 1/4 & 1/4 & 1/2 \end{bmatrix} \quad (\text{orden a, b, c})$$

Distribución estacionaria. Por simetría entre b y c: $\pi_b = \pi_c$. De $\pi_a = 3/4 \pi_a + 1/4 \pi_b + 1/4 \pi_c \Rightarrow 1/4 \pi_a = 1/2 \pi_b \Rightarrow \pi_a = 2\pi_b$. Con $\Sigma = 1$: $4\pi_b = 1 \Rightarrow \pi = (1/2, 1/4, 1/4)$.

Código compacto (Huffman) por estado (se asigna longitud 1 a la transición más probable):

Estado	$p(\cdot \text{estado})$	Código	Longitudes	$L_M(\text{estado})$	$H(\cdot \text{estado})$
a	$a \ 3/4 \cdot b \ 1/8 \cdot c \ 1/8$	$a \rightarrow 1, b \rightarrow 00, c \rightarrow 01$	1,2,2	$3/4 \cdot 1 + 1/8 \cdot 2 + 1/8 \cdot 2 = 5/4 = 10/8$	1.0613

b	a 1/4 · b 1/2 · c 1/4	a→10, b→0, c→11	2,1,2	1/4·2 + 1/2·1 + 1/4·2 = 3/2 = 12/8	1.5
c	a 1/4 · b 1/4 · c 1/2	a→10, b→11, c→0	2,2,1	1/4·2 + 1/4·2 + 1/2·1 = 3/2 = 12/8	1.5

Longitud de Markov:

$$L_M = \sum_i \pi_i L_M(s_i) = 1/2 \cdot 10/8 + 1/4 \cdot 12/8 + 1/4 \cdot 12/8 = 11/8 = \underline{1.375}$$

Longitud sin memoria. Si se ignora la dependencia y se codifica la fuente con sus probabilidades estacionarias (1/2, 1/4, 1/4), el Huffman es a→0, b→10, c→11:

$$L = 1/2 \cdot 1 + 1/4 \cdot 2 + 1/4 \cdot 2 = 3/2 = 12/8 = 1.5 (= H(S), \text{fuente diádica})$$

Conclusión: $L_M = 11/8 < 12/8 = L$ ✓.

Aprovechar la memoria ahorra 1/8 bit por símbolo (8.3 %).

Además $H(S_M) = 1/2(1.0613) + 1/4(1.5) + 1/4(1.5) = 1.2806 \leq L_M = 1.375 < L = H(S) = 1.5$, de modo que $H(S_M) \leq H(S)$ como exige la teoría.

Ejercicio 7 — Usando el texto del ejercicio 6, Práctico No 3

Texto: ERRECONERRECIGARROERRECONERREBARRIL (35 símbolos, 10 distintos). Código bloque de referencia: 4 bits/símbolo $\Rightarrow$ 140 bits originales.

Código bloque: A=0000, B=0001, C=0010, E=0011, G=0100, I=0101, L=0110, N=0111, O=1000, R=1001.

a) Codificar mediante los métodos: Huffman, Fano y Shannon como fuente de Markov de orden 1

Estadística de primer orden relevada (fila = símbolo anterior; n = veces que ocurre como estado con sucesor; el último símbolo L no tiene sucesor, por eso hay 34 transiciones):

Estado	apariciones	transiciones	p(siguiente estado)
A	2	2	R: 1
B	1	1	A: 1

C	3	3	I: 1/3 · O: 2/3
E	8	8	B: 1/8 · C: 3/8 · R: 1/2
G	1	1	A: 1
I	2	2	G: 1/2 · L: 1/2
L	1	0	— (último símbolo del texto)
N	2	2	E: 1
O	3	3	E: 1/3 · N: 2/3
R	12	12	E: 1/3 · I: 1/12 · O: 1/12 · R: 1/2

Longitud media por estado $L_M(s_i)$ en bits (los códigos por estado se construyen a mano; aquí solo el valor a verificar; 0 = transición determinista):

Estado	# sucesores	Huffman	Fano	Shannon
A	1	0.000	0.000	0.000
B	1	0.000	0.000	0.000

C	2	1.000	1.000	1.333
E	3	1.500	1.500	1.625
G	1	0.000	0.000	0.000
I	2	1.000	1.000	1.000
N	1	0.000	0.000	0.000
O	2	1.000	1.000	1.333
R	4	1.667	1.667	1.833

Longitudes medias de Markov $L_M = \sum p(s_i)L_M(s_i)$ con $p(s_i)$ = fracción de transiciones que salen del estado (n/34):

Método	Bits de datos (34 transiciones)	+ 1.er símbolo	Total	L_M (bits/símbolo)
Huffman	40	4	44	1.1765 (= 20/17)
Fano	40	4	44	1.1765
Shannon	45	4	49	1.3235 (= 45/34)

Entropías empíricas: $H(S) = 2.7713$ bits (sin memoria) y $H(S_M) = 1.1254$ bits (Markov 1.er orden) $\Rightarrow H(S_M) \leq L_M$ y $H(S_M) \ll H(S)$: el texto tiene mucha dependencia entre símbolos consecutivos. Huffman y Fano empatan (mismas longitudes en todos los estados); Shannon es peor por el redondeo hacia arriba.

b) Relevar para la fuente la estadística de primer orden y realizar una codificación predictiva.

Frecuencias de símbolos (orden 0): R: 12/35, E: 8/35, C: 3/35, O: 3/35, N: 2/35, I: 2/35, A: 2/35, G: 1/35, B: 1/35, L: 1/35. $H(S) = 2.7713$ bits.

Reglas PPM-C: en cada contexto U = símbolos distintos, total = suma de frecuencias; $P(x) = \text{freq}(x)/(\text{total} + U)$, $P(\text{ESC}) = U/(\text{total} + U)$ (el peso de ESC es virtual). Un contexto vacío emite ESC con probabilidad 1 (0 bits). O-1 es uniforme sobre el alfabeto de 10 símbolos (1/10). Costo ideal de cada evento = $-\log_2 P$ (lo que entrega el codificador aritmético, salvo el *flush*).

#	Contexto O1	Símbolo	Eventos [probabilidad]	Bits
1	–	E	E@O-1 [1/10]	3.322
2	E	R	ESC@O0 [1/2] → R@O-1 [1/10]	4.322
3	R	R	R@O0 [1/4]	2.000
4	R	E	ESC@O1 [1/2] → E@O0 [1/5]	3.322
5	E	C	ESC@O1 [1/2] → ESC@O0 [1/3] → C@O-1 [1/10]	5.907
6	C	O	ESC@O0 [3/8] → O@O-1 [1/10]	4.737
7	O	N	ESC@O0 [2/5] → N@O-1 [1/10]	4.644
8	N	E	E@O0 [1/6]	2.585
9	E	R	R@O1 [1/4]	2.000

10	R	R	R@O1 [1/4]	2.000
11	R	E	E@O1 [1/5]	2.322
12	E	C	C@O1 [1/5]	2.322
13	C	I	ESC@O1 [1/2] → ESC@O0 [5/17] → I@O-1 [1/10]	6.087
14	I	G	ESC@O0 [6/19] → G@O-1 [1/10]	4.985
15	G	A	ESC@O0 [1/3] → A@O-1 [1/10]	4.907
16	A	R	R@O0 [4/23]	2.524
17	R	R	R@O1 [1/3]	1.585
18	R	O	ESC@O1 [2/7] → O@O0 [1/25]	6.451
19	O	E	ESC@O1 [1/2] → E@O0 [2/13]	3.700
20	E	R	R@O1 [1/3]	1.585
21	R	R	R@O1 [1/3]	1.585
22	R	E	E@O1 [1/5]	2.322

23	E	C	C@O1 [2/7]	1.807
24	C	O	O@O1 [1/4]	2.000
25	O	N	N@O1 [1/4]	2.000
26	N	E	E@O1 [1/2]	1.000
27	E	R	R@O1 [3/8]	1.415
28	R	R	R@O1 [4/11]	1.459
29	R	E	E@O1 [1/4]	2.000
30	E	B	ESC@O1 [2/9] → ESC@O0 [8/37] → B@O-1 [1/10]	7.701
31	B	A	A@O0 [1/39]	5.285
32	A	R	R@O1 [1/2]	1.000
33	R	R	R@O1 [5/13]	1.379
34	R	I	ESC@O1 [3/14] → I@O0 [1/42]	7.615
35	I	L	ESC@O1 [1/2] → ESC@O0 [9/43] → L@O-1 [1/10]	6.578

Total ideal = 116.45 bits $\approx$ 3.327 bits/símbolo ($\approx$ 119 bits reales con el flush del aritmético). Es decir, PPM-C paga el aprendizaje: los símbolos nuevos cuestan entre 5 y 8 bits por los ESC y por el O-1.

Tabla O0 final: R:12, E:8, C:3, O:3, N:2, I:2, A:2, G:1, B:1, L:1 (ESC = U = 10).

c) Realizar una transformación Burrows-Wheller y codificar mediante cuenta de símbolos iguales (RLE).

Se agrega \$ (menor que cualquier letra) para que la transformada sea reversible:

ERRECONERRECIGARROERRECONERREBARRIL\$ (36 símbolos). Rotaciones cíclicas ordenadas (última columna = BWT; a la derecha, la rotación de origen):

0 ERRECONERRECIGARROERRECONERREBARRI[L](rot. 35)1ARRIL
ERRECONERRECIGARROERRECONERRE[B] (rot. 30)

2 ARROERRECONERREBARRILERRECONERRECI[G](rot. 14)3BARRIL
ERRECONERRECIGARROERRECONERRE[E] (rot. 29)

4 CIGARROERRECONERREBARRILERRECONERRE[E](rot. 11)5CONERREBARRIL
ERRECONERRECIGARROERRE[E] (rot. 22)

6 CONERRECIGARROERRECONERREBARRILERRE[E](rot. 4)7EBARRIL
ERRECONERRECIGARROERRECONER[R] (rot. 28)

8 ECIGARROERRECONERREBARRILERRECONER[R](rot. 10)9ECONERREBARRIL
ERRECONERRECIGARROER[R] (rot. 21)

10 ECONERRECIGARROERRECONERREBARRILER[R](rot. 3)11ERREBARRIL
ERRECONERRECIGARROERRECO[N] (rot. 25)

12 ERRECIGARROERRECONERREBARRILERRECO[N](rot. 7)13ERRECONERREBARRIL
ERRECONERRECIGARR[O] (rot. 18)

14 ERRECONERRECIGARROERRECONERREBARRIL[] (rot. 0)15GARROERRECONERREBARRIL
ERRECONERREC[I] (rot. 13)

16 IGARROERRECONERREBARRILERRECONERRE[C](rot. 12)17IL
ERRECONERRECIGARROERRECONERREBAR[R] (rot. 33)

18 LERRECONERRECIGARROERRECONERREBARR[I](rot. 34)19NERREBARRIL
ERRECONERRECIGARROERREC[O] (rot. 24)

20 NERRECIGARROERRECONERREBARRILERREC[O](rot. 6)21OERRECONERREBARRIL
ERRECONERRECIGAR[R] (rot. 17)

22 ONERREBARRILERRECONERRECIGARROERRE[C](rot. 23)23ONERRECIGARROERRECONERREBARRIL
ERRE[C] (rot. 5)

24 REBARRILERRECONERRECIGARROERRECON[R](rot. 27)25RECIGARROERRECONERREBARRIL
ERRECON[R] (rot. 9)

26 RECONERREBARRIL
ERRECONERRECIGARROE[R](rot. 20)27RECONERRECIGARROERRECONERREBARRILE[R] (rot. 2)

28 RILERRECONERRECIGARROERRECONERREBA[R](rot. 32)29ROERRECONERREBARRIL
ERRECONERRECIGA[R] (rot. 16)

30 RREBARRILERRECONERRECIGARROERRECON[E](rot. 26)31RRECIGARROERRECONERREBARRIL
ERRECON[E] (rot. 8)

32 RRECONERREBARRIL
ERRECONERRECIGARRO[E](rot. 19)33RRECONERRECIGARROERRECONERREBARRIL[E] (rot. 1)

34 RRILERRECONERRECIGARROERRECONERREB[A](rot. 31)35RROERRECONERREBARRIL
ERRECONERRECIG[A] (rot. 15)

BWT (última columna): **LBGEEEEERRRRNNO\$ICRIOORCCRRRRRREEEEAA**

RLE (cuenta de símbolos iguales, formato de los apuntes: la cuenta se antepone solo si la racha es mayor que 1): **LBG4E4R2N0\$ICRI2OR2C6R4E2A** → 18 rachas; se representa como 26 símbolos en vez de 36.

Rachas: L×1, B×1, G×1, E×4, R×4, N×2, O×1, \$×1, I×1, C×1, R×1, I×1, O×2, R×1, C×2, R×6, E×4, A×2.

Tamaño: cada racha como par (símbolo 4 bits + cuenta 3 bits, pues la racha máxima es 6) ⇒ 18×7 = **126 bits**. **Comparación:** RLE aplicado directamente al texto original da 29 rachas (solo **RR** se repite) ⇒ 29×(4+2) = 174 bits > 140: **expande**. La BWT es la que agrupa (**EEEE**, **RRRR**, **RRRRRR**, **EEEE**).

Reversibilidad: la BWT invertida (ordenar columnas y reconstruir) devuelve la fila que termina en \$, que es el texto original.

d) Obtener los porcentajes de compresión y comparar los resultados obtenidos.

Método	Bits	bits/símbolo	% compresión vs. bloque	Observación
Bloque (4 bits/símbolo)	140	4.00	0.0 %	referencia
Markov-1 + Huffman	44	1.26	68.6 %	necesita tabla de 17 transiciones

Markov-1 + Fano	44	1.26	68.6 %	idem
Markov-1 + Shannon	49	1.40	65.0 %	idem
PPM-C (orden ≤ 1)	116.5	3.33	16.8 %	adaptativo, sin tabla
BWT + RLE (pares 4+3 bits)	126	3.60	10.0 %	sin tabla
RLE directo (pares 4+2 bits)	174	4.97	-24.3 %	expande
LZW (ej. 6.b)	144	4.11	-2.9 %	códigos de 6 bits
LZ77 6/4 (ej. 6.a)	216	6.17	-54.3 %	tokens de 9 bits

Lectura de los resultados

1. **Markov + Huffman/Fano (44 bits)** es «el mejor» solo si la tabla es gratis. El modelo estático se calculó sobre este mismo texto; para descomprimir habría que enviar las 17 transiciones con sus códigos (al menos $17 \times (4+3) \approx 120$ bits de símbolo destino y longitud, más la estructura por estado). Con cabecera, el total supera los 140 bits: **expande**. Con textos cortos, la cabecera domina (es lo que mide la *Prueba 1* del práctico de máquina).
 2. **PPM-C** (116 bits, 16.8 % de ahorro real) no envía nada: aprende el modelo de a poco. Es, junto con BWT + RLE, un método sin costos ocultos que comprime frente al bloque, aunque poco por la longitud del texto (35 símbolos; cada símbolo nuevo cuesta entre 5 y 8 bits)).
 3. **BWT + RLE** mejora modestamente (10.0 %) porque la BWT junta **E, R, C...** pero hay solo 18 rachas para 36 símbolos; el RLE directo empeora. Con más texto las rachas crecen y la ganancia también.
 4. **Shannon** queda por detrás de Huffman/Fano (redondeo hacia arriba), y **LZ77/LZW** no rinden en un texto de 35 símbolos.
 5. Orden teórico coherente: $H(S_M) = 1.125 \leq L_M^{\text{(Huffman)}} = 1.176 \leq L_M^{\text{(Shannon)}} = 1.324 < H(S) = 2.771$ bits/símbolo.
-

Práctico 4

Ejercicio 1 — Comprobar que $H_r \leq L$

Teorema (cota inferior, apuntes). Para todo código instantáneo (o unívocamente decodificable) de base r sobre una fuente con probabilidades p_i y longitudes l_i :

$$H_r(S) \leq L = \sum_i p_i l_i.$$

Demostración (binaria, $r = 2$). Por la desigualdad de Jensen ($\ln x \leq x - 1$) y la inecuación de Kraft:

$$H - L = \sum p_i \cdot \log_2 (2^{(-l_i)} / p_i) \leq \log_2 (\sum p_i \cdot 2^{(-l_i)} / p_i) = \log_2 (\sum 2^{(-l_i)}) \leq \log_2 (1) = 0$$

La igualdad vale si y solo si $p_i = 2^{(-l_i)}$ (fuente diádica).

Comprobación numérica. Para cada conjunto se calcula H y se construye el código de Huffman (óptimo) y, como segundo código instantáneo de comparación, el de Shannon ($l_i = \lceil \log_2 1/p_i \rceil$). Se verifica $H \leq L_{\text{Huffman}} \leq L_{\text{Shannon}}$ y $L_{\text{Huffman}} < H + 1$.

Conjunto a) $p = (0.4, 0.2, 0.2, 0.1, 0.1)$

s_i	p_i	$-\log_2 p_i$	l_i Huffman	Código Huffman	l_i Shannon
s1	0.4	1.3219	2	11	2
s2	0.2	2.3219	2	00	3
s3	0.2	2.3219	2	01	3
s4	0.1	3.3219	3	100	4
s5	0.1	3.3219	3	101	4

- Entropía: $H = -\sum p_i \log_2 p_i = 2.1219$ bits.
- Huffman: $L_H = \sum p_i l_i = 2.20$; Kraft $\sum 2^{(-l_i)} = 1$ (código completo).
- Shannon: $L_S = 2.80$ (Kraft = $0.6250 < 1$, incompleto).

- **Comprobación:** $2.1219 \leq 2.20 \leq 2.80$ ✓ y $2.20 < H + 1 = 3.1219$ ✓. Eficiencia Huffman $\eta = 96.45 \%$.

Conjunto b) $p = (0.4, 0.4, 0.1, 0.1)$

s_i	p_i	$-\log_2 p_i$	l_i Huffman	Código Huffman	l_i Shannon
s1	0.4	1.3219	2	11	2
s2	0.4	1.3219	1	0	2
s3	0.1	3.3219	3	100	4
s4	0.1	3.3219	3	101	4

- Entropía: $H = -\sum p_i \log_2 p_i = 1.7219$ bits.
- Huffman: $L_H = \sum p_i l_i = 1.80$; Kraft $\sum 2^{-l_i} = 1$ (código completo).
- Shannon: $L_S = 2.40$ (Kraft = $0.6250 < 1$, incompleto).
- **Comprobación:** $1.7219 \leq 1.80 \leq 2.40$ ✓ y $1.80 < H + 1 = 2.7219$ ✓. Eficiencia Huffman $\eta = 95.66 \%$.

Conjunto c) $p = (0.4, 0.3, 0.2, 0.1)$

s_i	p_i	$-\log_2 p_i$	l_i Huffman	Código Huffman	l_i Shannon
s1	0.4	1.3219	1	0	2
s2	0.3	1.7370	2	10	2
s3	0.2	2.3219	3	111	3

s4	0.1	3.3219	3	110	4
----	-----	--------	---	-----	---

- Entropía: $H = -\sum p_i \log_2 p_i = 1.8464$ bits.
- Huffman: $L_H = \sum p_i l_i = 1.90$; Kraft $\sum 2^{-l_i} = 1$ (código completo).
- Shannon: $L_S = 2.40$ (Kraft = $0.6875 < 1$, incompleto).
- **Comprobación:** $1.8464 \leq 1.90 \leq 2.40$ ✓ y $1.90 < H + 1 = 2.8464$ ✓. Eficiencia Huffman $\eta = 97.18$ %.

Resumen

Conjunto	H_r (bits)	L_{Huffman}	L_{Shannon}	$\text{¿}H \leq L\text{?}$
a	2.1219	2.20	2.80	✓
b	1.7219	1.80	2.40	✓
c	1.8464	1.90	2.40	✓

Ningún código puede quedar por debajo de H ; Huffman es el que más se acerca (ninguno de los tres conjuntos es diádico, por eso $L > H$ estrictamente).

Ejercicio 2 — Fuente con cuatro eventos equiprobables: ¿es posible $L = 1.86$?

$p(s_i) = 1/4 \Rightarrow H = 4 \cdot 1/4 \log_2 4 = 2$ bits.

Respuesta: no. Dos razones independientes:

1. **Teorema de Shannon (cota inferior):** todo código unívocamente decodificable cumple $L \geq H = 2$. Como $1.86 < 2$, ningún código —Huffman, Shannon, Fano ni ningún otro— puede alcanzarlo.
2. **Kraft:** con p_i iguales, $L = 1/4(l_1 + l_2 + l_3 + l_4)$ solo toma valores múltiplos de $1/4$. El valor menor que 2 más cercano es 1.75, que corresponde a longitudes (1, 2, 2, 2) con $\sum 2^{-l_i} = 1/2 + 3/4 = 5/4 > 1$: viola Kraft, no existe código instantáneo. Además 1.86 no es múltiplo de $1/4$.

Mínimo valor de L : $L_{\min} = H = 2$ bits, con las cuatro longitudes iguales a 2 (los tres métodos coinciden):

s_i	p_i	Huffman = Fano = Shannon
-------	-------	--------------------------------

s1	1/4	00
s2	1/4	01
s3	1/4	10
s4	1/4	11

$L_H = L_F = L_S = 2$, $\eta = 1$, $R = 0$ (fuente diádica: $p_i = 2^{-2}$). Para extensiones de orden n vale lo mismo: $L \leq n$ y se alcanza.

Ejercicio 3 — Igual, con $p = (1/2, 1/4, 1/8, 1/8)$

$H = 1/2 \cdot 1 + 1/4 \cdot 2 + 2 \cdot 1/8 \cdot 3 = 1.75$ bits.

- **Cota de Shannon:** $L \geq H = 1.75$. Ahora $1.86 \geq 1.75$, así que el teorema **no** lo descarta ($H \leq 1.86 < H + 1$).
- **Pero no es alcanzable exactamente:** $L = 1/2 l_1 + 1/4 l_2 + 1/8(l_3 + l_4)$, o sea $8L = 4l_1 + 2l_2 + l_3 + l_4 \in \mathbb{Z}$. Con $L = 1.86$ daría $8L = 14.88 \notin \mathbb{Z}$. Con longitudes enteras, L solo puede valer múltiplos de $1/8$ (... , 1.75, 1.875, 2.0, ...): **1.86 no es posible para ninguna codificación** (sí lo serían 1.875, p. ej. longitudes (1, 2, 3, 4) con Kraft = $15/16 \leq 1$, código instantáneo pero no óptimo).

Mínimo valor de L: $L_{\min} = H = 1.75$, alcanzado porque la fuente es diádica ($p_i = 2^{-l_i}$, $l = (1, 2, 3, 3)$, Kraft = 1). Los tres métodos coinciden:

s_i	p_i	l_i	Huffman = Fano = Shannon
s1	1/2	1	0
s2	1/4	2	10

s3	1/8	3	110
s4	1/8	3	111

$L = 1/2 + 2/4 + 3/8 + 3/8 = 1.75$, $\eta = 1$, $R = 0$. Un código con $L = 1.875$ (el valor factible inmediatamente superior) tendría $\eta = 93.3\%$.

Ejercicio 4 — Código trinario (r = 3) para 7 símbolos

$p = (1/3, 1/3, 1/9, 1/9, 1/27, 1/27, 1/27)$, suma = $2/3 + 2/9 + 3/27 = 1$ ✓.

Entropía (base 3):

$$H_3(S) = 2 \cdot 1/3 \log_3 3 + 2 \cdot 1/9 \log_3 9 + 3 \cdot 1/27 \log_3 27 = 2/3 + 4/9 + 1/3 = 13/9 = 1.4444 \text{ trits}$$

(en bits: $H_2 = 13/9 \log_2 3 = 2.2894$ bits).

Huffman ternario (7 símbolos: $7 - 1 = 6$ es múltiplo de $r - 1 = 2 \Rightarrow$ no hacen falta símbolos ficticios).
 Resultado esperado:

s _i	p _i	Código	l _i
s1	1/3	0	1
s2	1/3	1	1
s3	1/9	20	2

s4	1/9	21	2
s5	1/2 7	220	3
s6	1/2 7	221	3
s7	1/2 7	222	3

Longitud media: $L = 1/3 + 1/3 + 2/9 + 2/9 + 9/27 = 2/3 + 4/9 + 1/3 = 13/9 = 1.4444$ trits/símbolo.

Kraft ternario: $2 \cdot 3^{-1} + 2 \cdot 3^{-2} + 3 \cdot 3^{-3} = 2/3 + 2/9 + 1/9 = 1$ ✓ (código completo).

Conclusión: $L = H_3 = 13/9 \Rightarrow \eta = 1$, $R = 0$: todas las p_i son potencias de $1/3$ (fuente diádica en base 3, $l_i = -\log_3 p_i$), por eso Huffman alcanza exactamente la cota.

Ejercicio 5 — Fuente $S = (s_1, s_2, s_3)$, $p = (1/2, 1/4, 1/4)$: Huffman, Shannon y Fano

$H = 1/2 \cdot 1 + 2 \cdot 1/4 \cdot 2 = 1.5$ bits.

s_i	p_i	Huffm an	Fano	Shann on
s1	1/2	0	0	0
s2	1/4	10	10	10

s3	1/4	11	11	11
----	-----	----	----	----

$L_H = 1.5, L_F = 1.5, L_S = 1.5$ bits/símbolo, los tres iguales a $H = 1.5 \Rightarrow \eta = 1, R = 0$ (fuente diádica).

Ejercicio 6 — Segunda y tercera extensión de la fuente anterior

Para la extensión de orden n : S^n tiene 3^n símbolos con $p(s_{i1} \dots s_{in}) = \prod p(s_{ik})$ y $H(S^n) = nH(S)$. El teorema de Shannon para extensiones da:

$$H(S) \leq L_{\square}/n < H(S) + 1/n.$$

Segunda extensión (9 símbolos)

Símbol o	p	l_i	Código (Huffman = Fano = Shannon)
s1s1	1/4	2	00
s1s2	1/8	3	010
s1s3	1/8	3	011
s2s1	1/8	3	100
s3s1	1/8	3	101
s2s2	1/16	4	1100
s2s3	1/16	4	1101
s3s2	1/16	4	1110

s3s3	1/16	4	1111
------	------	---	------

L_2 : Huffman = 3, Fano = 3, Shannon = 3 bits por par $\Rightarrow L_2/2 = 1.5$ bits/símbolo original. (Los tres métodos coinciden porque $-\log_2 p$ es entero para todo par.) Kraft: $1/4 + 4 \cdot 1/8 + 4 \cdot 1/16 = 1$ ✓.

Tercera extensión (27 símbolos)

Un símbolo con k letras distintas de s_1 tiene $p = 2^{-(3+k)}$, luego $l = 3 + k$ (todas las probabilidades son potencias de $1/2$). Agrupado por longitud:

# letras $\neq s_1$	Cantidad	p	l_i	Ejemplos	Σp
0	1	1/8	3	s1s1s1	1/8
1	6	1/16	4	s1s1s2, s1s3s1, ...	3/8
2	12	1/32	5	s1s2s3, s2s2s1, ...	3/8
3	8	1/64	6	s2s2s2, s3s2s3, ...	1/8

Suma de probabilidades = 1, Kraft = 1 (completo), $L_3 = 9/2 = 4.5$ bits por terna $\Rightarrow L_3/3 = 1.5$.

Tendencia de la longitud por símbolo de la fuente original

Orden n	$H(S^n) = nH$	$L_{\square} (H = F = S)$	$L_{\square}/n$	η
1	1.5	1.5	1.5	1

2	3	3	1.5	1
3	4.5	4.5	1.5	1

Conclusión. L/n es **constante e igual a $H = 1.5$** : la cota superior $H + 1/n$ del teorema de Shannon nunca se necesita porque ya con $n = 1$ el código es óptimo. Extender una fuente **diádica** no mejora nada (y sí complica: 3ⁿ símbolos y tablas mayores). La extensión solo reduce L/n hacia H cuando alguna p_i **no** es potencia de $1/2$ (ej.: en el ejercicio 1 del Práctico 4, $L = 2.2$ contra $H = 2.12$ para $n = 1$, y la brecha $< 1/n$ se cierra al extender).

Ejercicio 7 — PABLITOPABLITOPABLOCLAVO

Texto de 24 símbolos, alfabeto de $q = 9$ símbolos: A:4, L:4, O:4, B:3, P:3, I:2, T:2, C:1, V:1.

Código bloque de referencia: $\lceil \log_2 9 \rceil = 4$ bits/símbolo $\Rightarrow 24 \times 4 = \mathbf{96 \text{ bits}}$.

a) Segunda extensión codificada con Fano

Los símbolos de S^2 son pares de letras **consecutivos y no solapados**: PA BL IT OP AB LI TO PA BL OC LA VO (12 pares, 10 distintos). Probabilidad = frecuencia/12:

Par	Frecuencia	p	Código Fano	l
BL	2	1/6	010	3
PA	2	1/6	00	2
AB	1	1/12	100	3
IT	1	1/12	0110	4
LA	1	1/12	1110	4

LI	1	1/12	1010	4
OC	1	1/12	110	3
OP	1	1/12	0111	4
TO	1	1/12	1011	4
VO	1	1/12	1111	4

- $H(S^2) = 3.2516$ bits/par $\Rightarrow H(S^2)/2 = 1.6258$ bits/símbolo.
- $L_2 = \sum p_i = 3.3333$ bits/par $\Rightarrow L_2/2 = 1.6667$ bits/símbolo.
- Mensaje codificado: 12 pares $\Rightarrow$ **40 bits** (coincide con $12 \cdot L_2$ porque las probabilidades son las frecuencias empíricas).

b) Porcentaje de compresión y comparación con Fano como fuente de Markov de orden 1

Segunda extensión: $(1 - 40/96) \times 100 = 58.3 \%$.

Fano como fuente de Markov de orden 1. Se relevan las transiciones (23 en total) y se construye un código de Fano **por estado**; el primer símbolo (**P**) va en código bloque (4 bits).

Estado	Salidas	Probabilidades condicionadas	Código Fano (ϵ = palabra vacía, 0 bits)
A	4	B: 3/4, V: 1/4	$B \rightarrow 0, V \rightarrow 1$
B	3	L: 3/3	$L \rightarrow \epsilon$
C	1	L: 1/1	$L \rightarrow \epsilon$

I	2	T: 2/2	$T \rightarrow \epsilon$
L	4	I: 2/4, A: 1/4, O: 1/4	$I \rightarrow 0, A \rightarrow 11, O \rightarrow 10$
O	3	P: 2/3, C: 1/3	$P \rightarrow 0, C \rightarrow 1$
P	3	A: 3/3	$A \rightarrow \epsilon$
T	2	O: 2/2	$O \rightarrow \epsilon$
V	1	O: 1/1	$O \rightarrow \epsilon$

Solo los estados A (B/V), L (I/A/O) y O (P/C) tienen elección; en los demás el sucesor es único ($p = 1, 0$ bits).
 Bits del mensaje: 13 (transiciones) + 4 (primer símbolo) = **17 bits** $\Rightarrow (1 - 17/96) \times 100 = 82.3 \%$. Si se exigiera al menos 1 bit por transición: $25 + 4 = 29$ bits $\Rightarrow 69.8 \%$.

Entropía condicional empírica: $H_M = \sum_i p(s_i) H(S|s_i) = 0.5217$ bits/símbolo (contra $H(S) = 3.0221$).

Método	Bits	bits/símbolo	% compresión (vs. 96)	Tabla a transmitir
Código bloque	96	4	0 %	—
2. ^a extensión + Fano	40	1.667	58.3 %	10 pares con su código
Markov-1 + Fano	17	0.708	82.3 %	13 transiciones (7 con código)

Markov-1 + Fano (≥ 1 bit)	29	1.208	69.8 %	ídem
---------------------------------	----	-------	--------	------

El modelo de Markov comprime **mucho más** (17 contra 40 bits).

c) Fundamento de la diferencia

1. **La extensión de orden 2 es un modelo sin memoria de bloques.** Trata cada par como un símbolo independiente de los demás: solo captura la dependencia *dentro* del par y, como los pares están alineados, ignora la dependencia entre el 2.º símbolo de un par y el 1.º del siguiente (ej.: PA|BL, BL|IT). Su límite es $H(S^2)/2 = 1.626$ bits/símbolo.
2. **Markov de orden 1 condiciona cada símbolo en el anterior**, en todas las posiciones. Como el texto es casi determinista (B→L, P→A, I→T, T→O, V→O, C→L siempre; solo A, L, O ofrecen elección), la incertidumbre por símbolo cae a $H_M = 0.522$ bits, mucho menor que $H(S) = 3.022$. En términos de la teoría: $H(S^2)/2 \geq H_M$ (condicionar reduce entropía), y $H(S^n)/n \rightarrow H_M$ solo cuando $n \rightarrow \infty$.
3. **Costo del modelo.** Ambos métodos usan las estadísticas del propio texto, de modo que están "sobreajustados" a un mensaje de 24 símbolos. La tabla de Markov (13 transiciones) es algo más grande que la de la extensión (10 pares) y **no está incluida** en los bits; con textos cortos esa cabecera puede anular la ganancia. En un texto largo (con estadística estable) la ventaja de Markov se mantiene y el costo de tablas se amortiza.
4. La extensión de mayor orden n podría acercarse a H_M , pero al precio de q^n símbolos y de tablas crecientes; Markov logra el mismo efecto con q^2 probabilidades.

Ejercicio 8 — Pipeline BWT → RLE → Huffman sobre TOMATOMA\$

Cadena de 13 símbolos, alfabeto de 5 símbolos (T, O, M, A $\times 3$ y \$ $\times 1$). Código bloque de referencia: 3 bits $\Rightarrow 13 \times 3 = 39$ bits. Huffman directo (referencia) $\Rightarrow 30$ bits (23.1 % de compresión).

a) RLE directo

Rachas: T $\times 1$, O $\times 1$, M $\times 1$, A $\times 1$, T $\times 1$, O $\times 1$, M $\times 1$, A $\times 1$, T $\times 1$, O $\times 1$, M $\times 1$, A $\times 1$, \$ $\times 1$. Hay 13 rachas de longitud 1 para 13 símbolos: **no existe ninguna repetición contigua**. Con el formato de los apuntes (la cuenta solo se antepone si la racha es mayor que 1) la salida es idéntica a la entrada, TOMATOMA\$, con 0 % de compresión; si se codificara cada racha como par (símbolo, cuenta) el resultado sería el doble (26 símbolos): **RLE expande**. El texto repite *patrones* (TOMA) pero no *símbolos iguales seguidos*.

b) BWT y luego RLE

Rotaciones cíclicas ordenadas (\$ es el menor símbolo); la última columna es la salida de la BWT:

0 TOMATOMATOM[A](rot. 12)1ATOMATOMATO[M] (rot. 11)

2 ATOMATOMATO[M](rot. 7)3ATOMATOMATO[M] (rot. 3)

4 MATOMATOMAT[O](rot. 10)5MATOMATOMAT[O] (rot. 6)

6 MATOMATOMAT[O](rot. 2)7OMATOMATOMA[T] (rot. 9)

8 OMATOMATOMA[T](rot. 5)9OMATOMATOMA[T] (rot. 1)

10 TOMATOMATOM[A](rot. 8)11TOMATOMATOM[A] (rot. 4)

12 TOMATOMATOMA[\$] (rot. 0)

BWT = AMMMOOOTTTAA\$ (la transformada **agrupa** símbolos iguales, porque **MA**, **OM**, **TO** se repiten y los contextos ordenados quedan juntos).

RLE de la BWT: rachas A×1, M×3, O×3, T×3, A×2, \$×1 ⇒ A3M3O3T2A\$ (10 símbolos en vez de 13; 6 rachas en vez de 13).

Huffman sobre la salida del RLE (los dígitos son cuentas, no hay ambigüedad porque el alfabeto original no tiene dígitos): alfabeto de 7 símbolos.

Total: **27 bits**.

Verificación: la BWT inversa (agregar la columna, ordenar, repetir 13 veces) devuelve como fila terminada en \$ la cadena TOMATOMATOMA\$ ✓.

Comparación

Método	Bits	bits/símbolo original	% compresión
Bloque (3 bits)	39	3.00	0 %
RLE directo (sin Huffman)	13 símbolos → 39	3	0 % (no hay rachas)
Huffman directo	30	2.31	23.1 %
BWT + Huffman (sin RLE)	30	2.31	23.1 %
BWT + RLE + Huffman	27	2.08	30.8 %

Lectura.

- **RLE directo no aporta** (0 rachas); la BWT lo hace útil: pasa de 13 a 6 rachas, y de 13 a 10 símbolos.
- **BWT sola + Huffman no mejora** (30 bits = igual que Huffman directo): la BWT es una permutación, no cambia las frecuencias, y Huffman de orden 0 no ve el orden. La ganancia aparece únicamente cuando el orden se explota con RLE.
- **BWT + RLE + Huffman: 27 bits contra 30 de Huffman directo** (-3 bits, 10.0 % menos). La mejora es modesta porque la cadena es corta y el RLE introduce símbolos nuevos (dígitos 3, 2) que agrandan el alfabeto (7 símbolos contra 5). En textos largos con contextos repetidos las rachas son más largas y la ganancia crece (es el esquema de **bzip2**).
- Costos no incluidos: tabla de Huffman (7 entradas contra 5) y posición/marca de fin (\$) para invertir la BWT.